

# **RELATÓRIO DE PERFORMANCE - COMPARAÇÃO DE ALGORITMOS DE ORDENAÇÃO**

YURI SAMUEL SOARES BATISTA  
PEDRO HENRIQUE RODRIGUES TRINDADE DA SILVA

VESPASIANO, 2026

# 1. INTRODUÇÃO

O problema de ordenação consiste em reorganizar os elementos de uma sequência (array, lista, vetor) em uma ordem específica, geralmente crescente ou decrescente. Este é um dos problemas fundamentais da Ciência da Computação, com aplicações em diversas áreas como busca de dados, otimização de algoritmos, compressão de informações e organização de bancos de dados.

A eficiência de um algoritmo de ordenação é medida principalmente pelo número de comparações e trocas realizadas entre os elementos, expresso através da notação Big-O. Este trabalho compara dois algoritmos de complexidades distintas: **Insertion Sort ( $O(n^2)$ )**, considerado um algoritmo simples, e **Quick Sort ( $O(n \log n)$ )**, classificado como eficiente. A análise é feita para diferentes cenários (melhor caso, pior caso e caso médio) e tamanhos de entrada (1.000 e 1.000.000 de elementos).

---

## 2. FUNCIONAMENTO DOS ALGORITMOS

### 2.1 Insertion Sort ( $O(n^2)$ )

O Insertion Sort funciona de maneira semelhante à forma como um jogador de cartas organiza suas mãos. O algoritmo percorre o array da esquerda para a direita, e cada elemento é "inserido" na posição correta entre os elementos já ordenados à sua esquerda.

#### Pseudocódigo:

text

para i de 1 até n-1:

    chave = array[i]

    j = i - 1

    enquanto j >= 0 e array[j] > chave:

        array[j + 1] = array[j]

        j = j - 1

    array[j + 1] = chave

### Exemplo visual com [5, 2, 4, 6, 1, 3]:

text

Passo 1: [5, 2, 4, 6, 1, 3] → insere 2 → [2, 5, 4, 6, 1, 3]

Passo 2: [2, 5, 4, 6, 1, 3] → insere 4 → [2, 4, 5, 6, 1, 3]

Passo 3: [2, 4, 5, 6, 1, 3] → insere 6 → [2, 4, 5, 6, 1, 3]

Passo 4: [2, 4, 5, 6, 1, 3] → insere 1 → [1, 2, 4, 5, 6, 3]

Passo 5: [1, 2, 4, 5, 6, 3] → insere 3 → [1, 2, 3, 4, 5, 6]

## 2.2 Quick Sort ( $O(n \log n)$ )

O Quick Sort utiliza a estratégia "dividir para conquistar". O algoritmo escolhe um elemento chamado pivô, particiona o array de forma que elementos menores que o pivô fiquem à esquerda e maiores à direita, e então ordena recursivamente cada partição. Para evitar estouro de pilha em grandes entradas, foi utilizada uma versão **iterativa** com pilha gerenciada manualmente.

### Pseudocódigo da versão iterativa:

text

procedimento quickSortIterativo(array):

    pilha = nova Pilha()

    pilha.empilhar(0, tamanho-1)

    enquanto pilha não vazia:

        inicio, fim = pilha.desempilhar()

        se inicio < fim:

            pivo = particionar(array, inicio, fim)

            pilha.empilhar(inicio, pivo-1)

            pilha.empilhar(pivo+1, fim)

procedimento particionar(array, inicio, fim):

    pivo = array[fim]

    i = inicio - 1

    para j de inicio até fim-1:

        se array[j] <= pivo:

            i = i + 1

        trocar array[i] com array[j]

    trocar array[i+1] com array[fim]

    retornar i + 1

**Exemplo visual com [5, 2, 4, 6, 1, 3] (pivô = 3):**

text

Original: [5, 2, 4, 6, 1, 3]

Particiona: [2, 1, 3, 6, 4, 5] (3 na posição correta)

|        |           |
|--------|-----------|
| ↓      | ↓         |
| [2, 1] | [6, 4, 5] |
| ↓      | ↓         |
| [1, 2] | [4, 5, 6] |

Final: [1, 2, 3, 4, 5, 6]

---

### 3. RESULTADOS

Tabela Comparativa de Tempos de Execução

| Caso        | Tamanho   | Insertion Sort (ms)        | Quick Sort (ms) | Diferença (x vezes)             |
|-------------|-----------|----------------------------|-----------------|---------------------------------|
| Melhor Caso | 1.000     | 0,02                       | 0,88            | Quick Sort foi 44x mais lento   |
| Melhor Caso | 1.000.000 | 6,03                       | 128,08          | Quick Sort foi 21x mais lento   |
| Pior Caso   | 1.000     | 1,31                       | 0,17            | Insertion foi 7,7x mais lento   |
| Pior Caso   | 1.000.000 | 384.635,24 (≈ 6,4 minutos) | 126,27          | Insertion foi 3.046x mais lento |
| Caso Médio  | 1.000     | 0,22                       | 0,06            | Insertion foi 3,7x mais lento   |
| Caso Médio  | 1.000.000 | 83.618,13 (≈ 1,4 minutos)  | 96,58           | Insertion foi 866x mais lento   |

#### Análise dos Resultados

##### Melhor Caso (dados já ordenados):

- Insertion Sort teve desempenho excepcional: apenas 0,02ms para 1.000 e 6,03ms para 1.000.000 elementos
- Isso ocorre porque no melhor caso o Insertion Sort tem complexidade  $O(n)$ , fazendo apenas uma varredura sem realizar trocas
- Quick Sort, mesmo na versão iterativa, teve desempenho pior neste cenário específico

##### Pior Caso (dados em ordem decrescente):

- Insertion Sort teve desempenho catastrófico: **384 segundos (6,4 minutos)** para 1.000.000 elementos
- Isso representa a complexidade quadrática  $O(n^2)$  máxima: aproximadamente 500 bilhões de operações

- Quick Sort manteve desempenho consistente de ~126ms, cerca de **3.046 vezes mais rápido**

#### Caso Médio (dados aleatórios):

- Insertion Sort levou 83,6 segundos para 1.000.000 elementos
  - Quick Sort completou em apenas 96,58ms, sendo **866 vezes mais rápido**
- 

## 4. ANÁLISE CRÍTICA

### 4.1 Por que o algoritmo $O(n^2)$ demorou muito mais no arquivo grande?

O Insertion Sort apresenta complexidade quadrática  $O(n^2)$ , o que significa que o número de operações cresce proporcionalmente ao quadrado do tamanho da entrada. Analisando os resultados obtidos:

#### Para o pior caso (dados decrescentes):

- Com  $n = 1.000$ : 1,31 ms
- Com  $n = 1.000.000$ : 384.635,24 ms

O aumento no tempo foi de aproximadamente **293.000 vezes**, enquanto o aumento teórico de operações é de  $(1.000.000^2 / 1.000^2) = 1.000.000$  vezes. A diferença se deve ao fato de que para  $n=1.000$ , parte do tempo é overhead de leitura e chamadas de método, que se torna proporcionalmente menor para  $n=1.000.000$ .

#### Para o caso médio (dados aleatórios):

- Com  $n = 1.000$ : 0,22 ms
- Com  $n = 1.000.000$ : 83.618,13 ms

Aumento de aproximadamente **380.000 vezes**, confirmando o comportamento quadrático esperado.

**Conclusão prática:** Para 1 milhão de elementos, o Insertion Sort no pior caso leva **mais de 6 minutos**, enquanto o Quick Sort leva apenas **126 milissegundos**. Isso demonstra que algoritmos quadráticos são completamente inviáveis para grandes volumes de dados.

### 4.2 O que acontece com o tempo do algoritmo eficiente quando aumentamos a entrada?

O Quick Sort apresenta complexidade  $O(n \log n)$ , que cresce de forma muito mais suave. Analisando os resultados:

### Pior caso do Quick Sort:

- $n = 1.000$ : 0,17 ms
- $n = 1.000.000$ : 126,27 ms
- Aumento de **743 vezes** para 1.000x mais elementos

### Caso médio do Quick Sort:

- $n = 1.000$ : 0,06 ms
- $n = 1.000.000$ : 96,58 ms
- Aumento de **1.610 vezes** para 1.000x mais elementos

Teoricamente, o aumento deveria ser aproximadamente:

text

$$(n_2 \times \log_2 n_2) / (n_1 \times \log_2 n_1) = (1.000.000 \times 20) / (1.000 \times 10) = 2.000 \text{ vezes}$$

Os resultados obtidos (743x a 1.610x) estão dentro da ordem de grandeza esperada, considerando variações de hardware e otimizações da JVM.

### 4.3 Comportamento do Insertion Sort no melhor caso

Um resultado notável é que o Insertion Sort **superou o Quick Sort no melhor caso** (dados já ordenados). Com 1.000.000 de elementos, o Insertion Sort levou apenas 6,03 ms contra 128,08 ms do Quick Sort. Isso ocorre porque:

- No melhor caso, o Insertion Sort tem complexidade  $O(n)$  - apenas verifica cada elemento e não faz trocas
- O Quick Sort, mesmo no melhor caso, precisa realizar aproximadamente  $n \log_2 n$  operações de particionamento
- Para dados já ordenados, a versão com pivô no final ou no meio ainda precisa processar recursivamente todo o array

#### 4.4 Resumo da Análise

| Cenário                | Insertion Sort       | Quick Sort                  | Vencedor       |
|------------------------|----------------------|-----------------------------|----------------|
| Melhor caso (ordenado) | Excelente ( $O(n)$ ) | Bom ( $O(n \log n)$ )       | Insertion Sort |
| Caso médio (aleatório) | Ruim ( $O(n^2)$ )    | Excelente ( $O(n \log n)$ ) | Quick Sort     |
| Pior caso (decrecente) | Péssimo ( $O(n^2)$ ) | Bom ( $O(n \log n)$ )       | Quick Sort     |

### 5. CONCLUSÃO

Os resultados experimentais confirmam a análise teórica de complexidade:

1. **Para pequenas entradas (1.000 elementos)**, ambos os algoritmos têm desempenho aceitável, com diferenças imperceptíveis para o usuário.
2. **Para grandes entradas (1.000.000 elementos):**
  - O Insertion Sort se torna inviável no pior caso (6 minutos) e muito lento no caso médio (1,4 minutos)
  - O Quick Sort mantém tempos excelentes (cerca de 100ms) em todos os cenários, exceto melhor caso onde ainda é muito bom
3. **Recomendação prática:**
  - Para dados pequenos (até alguns milhares) ou dados já ordenados, o Insertion Sort pode ser uma escolha simples e eficiente
  - Para dados grandes ou não ordenados, algoritmos  $O(n \log n)$  como Quick Sort são **essenciais**
4. **Aprendizado importante:** A escolha do algoritmo de ordenação tem impacto DRAMÁTICO no desempenho para grandes volumes de dados. A diferença entre 6 minutos e 0,1 segundo (3.600x) demonstra por que a análise de complexidade de algoritmos é fundamental na ciência da computação.